

Terraform Foundations

Condensed study notes • Terraform Associate preparation

1. TERRAFORM & INFRASTRUCTURE AS CODE

Terraform is HashiCorp's Infrastructure-as-Code tool. It uses a **declarative** model: describe the desired end state and Terraform determines the required changes.

HCL (HashiCorp Configuration Language) describes resources and relationships. **IaC** defines infrastructure in version-controlled text instead of manual console actions.

Core capabilities: codify infrastructure • standardize workflows • reuse modules • work across providers such as AWS, Azure, Google Cloud, Kubernetes and GitHub.

2. KEY BENEFITS

Version control Track changes, audit history, troubleshoot and roll back.	Automation Reduce repetitive manual work and human error.	Scalability Change code to scale resources and manage environments.	Collaboration Use branches, pull requests, reviews and modules.
---	---	---	---

3. DECLARATIVE vs IMPERATIVE

Declarative — Terraform	Imperative — scripts/CLI
Describe the desired final state.	Describe the exact steps to perform.
Terraform builds a dependency graph and computes a plan.	You generally control order, retries and procedural logic.
State + plan/apply provide predictable reconciliation.	Idempotency/change handling must be managed by the script/tool.

4. TERRAFORM CORE COMPONENTS

Terraform Core CLI engine: parses configuration, builds the dependency graph, compares configuration with state, creates plans and orchestrates provider calls.	Providers Plugins translating Terraform declarations into API interactions with platforms such as AWS, Azure, GCP and GitHub.
Resources Declare infrastructure Terraform manages. They accept arguments and expose attributes.	State Maps configuration to real-world objects and stores IDs/metadata, enabling incremental planning.
Modules Reusable Terraform configuration packages with inputs and outputs.	Dependencies References let Terraform infer ordering; depends_on can enforce explicit ordering.

5. HCL BASICS

Common blocks: resource • data • variable • output • locals • module • provider • terraform

Comments: #, //, /* ... */

Files: Terraform automatically loads .tf files in the same directory as one configuration.

Style: use snake_case, two-space indentation, logical grouping and terraform fmt.

6. RESOURCE REFERENCING

Resource addresses use resource_type . resource_name, e.g. aws_vpc.vpc.

References avoid hard-coding, pass values between blocks and let Terraform infer dependencies.

Data sources query existing infrastructure/provider information and expose values for other configuration.

Resource names must be unique within a resource type in a module.

7. REUSABILITY, FILES & GIT

Variables: reuse the same configuration with different inputs.

Modules: package reusable infrastructure patterns; expose inputs and outputs.

Multiple .tf files: Terraform treats them as one configuration; split files logically (provider, networking, compute, etc.).

Git workflow: feature branch → pull request → review → CI checks such as terraform fmt, terraform validate, and terraform plan.

8. ESSENTIAL WORKFLOW

WRITE main.tf Declare resources	INIT terraform init Initialize	PLAN terraform plan Preview	APPLY terraform apply Execute	DESTROY terraform destroy Remove
--	---	--	--	---

9. EXAM / REAL-WORLD TAKEAWAYS

- **Declarative:** define what infrastructure should look like; Terraform determines how to reach it.
- **HCL:** configuration language for infrastructure and relationships.
- **Providers:** connect Terraform Core to platforms/APIs.
- **Resources:** managed infrastructure; **data sources:** read existing information.
- **State:** maps configuration to real infrastructure and enables incremental changes; protect it and use secure remote backends for shared environments.
- **References:** create dependencies and help determine operation order.
- **Modules + variables:** enable reuse and environment-specific configuration.
- **terraform fmt:** keeps HCL formatting consistent.
- Never commit secrets or sensitive state to version control.